

Project Work Plan: Hybrid Network Anomaly Detection

A step-by-step roadmap and technical glossary for building a robust C/Python network security tool, optimized for an Apple Silicon (M1) environment.

Phase 1: Workspace & Environment Setup (M1 Specific)

Since you are using a MacBook Air M1 (ARM64 architecture), you are running macOS (Darwin kernel). While macOS is great, industry-standard low-level networking (especially if you eventually touch eBPF) heavily relies on the Linux Kernel. Therefore, we will set up a Linux environment on your Mac.

1. Virtualization (Your Sandbox)

Do not develop the low-level C sniffer directly on macOS to avoid system instability. Instead:

- Download and install OrbStack (highly recommended for M1, lighter than Docker) or UTM.
- Spin up a lightweight Ubuntu ARM64 virtual machine (VM).
- This VM will be your dedicated development environment.

2. IDE & Toolchain

- IDE: Visual Studio Code (Ensure you download the "Apple Silicon" version).
- VS Code Extensions: Install "Remote - SSH" to code natively inside your Ubuntu VM, "C/C++", and "Python".
- C Dependencies (Run in VM terminal): `sudo apt update && sudo apt install build-essential libpcap-dev`
- Python Environment: We will use a virtual environment to keep packages clean.

3. Directory Structure

Open your VM terminal and run the following to create your folders:

```
$ mkdir -p ~/Projects/Network-Anomaly/{c_sniffer,ml_python,dataset,docs}
$ cd ~/Projects/Network-Anomaly
```

Phase 2: Developing the C Sniffer (Data Capture)

In this phase, you will write the C program that listens to the network interface.

1. Start by writing a simple C program that uses libpcap to open a network interface.
2. Set up a packet processing callback function.
3. Parse the IP and TCP/UDP headers using C structs and pointers.
4. Extract ONLY the metadata you need (e.g., packet length, timestamp, protocol type, TCP flags).

5. Output this data to the standard output (stdout) or write it to a named pipe (FIFO) for Python to read.

Phase 3: Python & Machine Learning (The Seminar)

This is where your seminar research comes to life.

1. Navigate to your ml_python folder: `cd ~/Projects/Network-Anomaly/ml_python`
2. Create a virtual environment: `python3 -m venv venv`
3. Activate it: `source venv/bin/activate`
4. Install ML libraries: `pip install pandas numpy scikit-learn`
5. Write a Python script that reads the parsed metadata generated by your C sniffer.
6. Implement an anomaly detection model (e.g., Isolation Forest or a basic Autoencoder) to flag unusual traffic patterns in real-time.

Essential Technical Glossary

As you begin your research and development, you will encounter these core concepts. Keep this reference handy.

libpcap:

A portable C/C++ library for network traffic capture. It is the underlying engine for tools like Wireshark and tcpdump.

OSI Model & Headers:

Network data is wrapped in layers. When capturing a packet, you will use C pointers to "peel" the Ethernet header, then the IP header, and finally the TCP/UDP header to read the metadata.

Network vs. Host Byte Order (Endianness):

Networks transmit data in Big-Endian format, while your M1 Mac uses Little-Endian. You must use functions like `ntohs()` (Network To Host Short) in C to convert the data before reading it.

Zero-Copy:

An advanced memory management technique where the CPU does not copy data from one memory area to another. This is crucial for high-performance network sniffers.

IPC (Inter-Process Communication):

The mechanisms used to let your C program talk to your Python script. Common methods include Named Pipes (FIFO), Sockets, or Shared Memory.

Feature Extraction:

In Machine Learning, this means transforming raw data (network packets) into numerical values (features) that an algorithm can understand (e.g., calculating the average time between packets).

Unsupervised Learning:

A branch of ML where the model finds patterns without being explicitly trained on labeled "attack" data. Perfect for anomaly detection, as it flags anything that deviates from "normal" traffic.